

# ME/CS/EE 133b Final Project: Stealth Robot

Mohammad Arbab, Ashiria Goel, Emily Xu

March 18, 2026

<https://drive.google.com/file/d/1n1FtCg3lHwls5h32X4x04Z7u2EG-Hhq/view?usp=sharing>

## 1 Overview

This project addresses the problem of stealth path planning in a dynamic environment, where a robot must navigate from a start location to a goal while avoiding detection from rotating cameras. Unlike classical motion planning problems that consider only static obstacles, this setting introduces time as a critical dimension. The robot must not only avoid collisions with walls but also ensure that it is never within the field of view of any camera at any point during its motion. As a result, the planning problem is formulated in a three-dimensional state space  $(x,y,t)$ , where valid trajectories must satisfy both spatial constraints and time-dependent visibility constraints.

To solve this problem, two sampling-based planning approaches were explored. One approach uses a Probabilistic Roadmap (PRM) extended into the temporal domain, where nodes are sampled in  $(x,y,t)$  and edges are constructed based on dynamic feasibility and continuous-time safety with respect to camera visibility. A graph is built offline, and A\* search is used to find a minimum-time path to the goal. In parallel, a Rapidly-exploring Random Tree (RRT)-based method was implemented, which incrementally builds a tree in the same spatiotemporal space by extending toward randomly sampled states while enforcing visibility and motion constraints. We experimented with two different RRT-based algorithms. One RRT-based method grows the tree in the spatial domain, producing a collision-free geometric path through the maze. Then, it incorporates time information in the post-processing step by assigning traveling times and inserting wait intervals whenever a segment interacts a camera's view. The second RRT-based method integrates timing into the tree expansion process. For each sampled target in space  $(x,y)$ , it evaluates candidate parent nodes according to travel time and potential wait time, inserts wait nodes, and uses RRT\* to improve the time-cost of the tree.

We compare three different strategies for stealth planning: an offline-based PRM planner, RRT with temporal consideration post-processing, and spatiotemporal RRT planner. Together, these approaches demonstrate different strategies for solving time-dependent stealth planning problems, highlighting the trade-offs between roadmap-based and tree-based exploration in dynamic environments.

## 2 Approach

### 2.1 Temporal Probabilistic Roadmap

To address the time-dependent stealth planning problem, we extend the classical Probabilistic Roadmap (PRM) framework into a spatiotemporal setting. Rather than planning solely in spatial

coordinates, the robot’s state is defined in a three-dimensional space  $(x,y,t)$ , where time captures the evolving visibility of rotating cameras. This allows the planner to explicitly reason about when a robot occupies a location, ensuring that all candidate trajectories remain collision-free and avoid detection over time.

The roadmap is constructed by randomly sampling nodes in  $(x,y,t)$  that lie in free space and are not visible to any camera at their associated time. Edges are then formed between nearby nodes based on dynamic feasibility and safety constraints. Specifically, edges must respect the robot’s constant-speed motion model, meaning the time difference between nodes must match the required travel time between their spatial positions. In addition to motion edges, the roadmap also allows “wait” edges, enabling the robot to remain stationary and exploit temporal gaps in camera coverage. Each candidate edge is validated by continuously interpolating between the two states and verifying that the robot remains collision-free and outside all camera fields of view throughout the entire transition.

Once the roadmap is constructed, the planning problem is reduced to a graph search. Multiple goal nodes are instantiated at different times to account for periodic camera motion, and A\* search is used to find a minimum-time path from the start to any valid goal. The cost of each edge corresponds to elapsed time, and the heuristic is based on the spatial distance to the goal scaled by the robot’s speed. This formulation enables efficient computation of feasible trajectories that balance spatial movement and timing, allowing the robot to coordinate its motion with the dynamic sensing environment.

## 2.2 Temporal RRT

This approach decouples the spatial and temporal aspect of the planning problem into two stages. The first stage is a standard RRT that is used to grow a tree solely in the spatial domain ( $t = 0$ ), treating the cameras as static. The planner is an ordinary RRT, where random points in  $(x,y)$  are sampled and the tree extends towards each sampled node using the closest neighbor to that node. The edges are then validated geometrically, ensuring that there is no collisions with the walls or obstacles. Afterwards, a post-processing occurs, simplifying the path and skipping unnecessary segments. This results in a geometric path from the start goal to the node with no consideration of the cameras or their timing.

In the second stage, time is incorporated during another post-processing pass over the simplified-path. Each path segment is signed a time based on the robot’s constant speed and the segment’s length. The planner then goes through each segment and checks at each point along it whether the robot will be in the view of the camera at the corresponding time. If the robot is in the camera field of view, then the robot will wait at a safe position, allowing the camera to rotate far enough until it’s safe for the robot to move along the segment.

The main advantage of this approach is simplicity. The spatial planning and temporal addition are separate, allowing for fast computation for the RRT without any major modification. The timing decisions are handled in a post-processing step. However, this also has its downsides as the waiting component is not optimal. The geometric path is essentially committed to before any timing is considered, so even though if a segment is collision-free and acceptable, if a camera’s field of view sweeps across it for an extended period of time, then the planner has no option but to wait.

## 2.3 Spatiotemporal RRT

To more tightly integrate time into the RRT planning process, we implement a temporal variant of RRT\*. We experimented with sampling in the  $(x,y,t)$  state similar to the temporal PRM, but

in RRT, a node’s time is naturally tied to the specific parent path used to reach it. As a result, time associated with a node is not an independent coordinate/state that can be sampled, but it is determined by the parent node, travel time, and waiting. In PRM, nodes can exist independently (in  $(x,y,t)$  space) because the roadmap is a graph of possible states – time consistency is only enforced at the step when edges are added. In RRT, we are building a tree of realized trajectories, where time is fundamentally path-dependent.

In RRT, each node has one parent. A node is not just an  $(x,y)$  location in space, but it needs to encode the time when the robot reaches the location, and the specific trajectory required to get there. So in our algorithm, when we sample a node  $(x,y)$ , its time of arrival at that node cannot be chosen arbitrarily. It depends on which parent it is connected to, how long the robot waited there, and how long it took to move from the parent to the node. In our RRT\* implementation, we decide to use time as a "cost-to-come" quantity. If the same spatial point were reached from a different parent, it might need a different arrival time to avoid the cameras.

Each node maintains an arrival time and accumulated cost (total elapsed time from start node). For each sampled spatial target, the planner considers nearby nodes in a limited radius in the existing tree, and evaluates the cost of reaching the new position from each candidate parent. The cost is computed using time, taking into account how long the robot must wait at the parent node before collision-free and stealth-free motion is possible. The algorithm incrementally searches over possible departure times from a candidate parent node, and checks if the motion segment avoids obstacles and cameras. The total edge cost is the sum of the required waiting time and the physical travel time.

We note that yes, this doesn’t directly use distance as the main cost evaluation metric. Instead, the objective is to minimize total elapsed time, which includes both travel time and any waiting required to avoid camera visibility. This introduces a trade-off between path length and timing. A shorter path may require significant waiting if it passes through an area cameras frequent, whereas a slightly longer (distance) path that avoids/loops can be traversed with little to no delay. Shorter paths are not always more optimal, in some cases a slightly longer path with less waiting will result in a lower cost/more intuitive path. The optimal trajectory is determined by both geometry and timing, rather than just distance.

The parent node is selected by minimizing the total cost-to-get there, with waiting time. If waiting is required, a 'wait' node is inserted before the next motion. After adding a new node, nearby nodes are checked to determine whether a lower-cost path exists through the new node.

This allows the planner to incorporate timing without sampling in the  $(x,y,t)$  state. By incorporating waiting as an option between edge evaluation and cost computations, the algorithm produces trajectories that are physically feasible (collision-free) and stealth in time.

We use discretized time to a finite set of samples over one camera rotation period. We tried continuous-time visibility, but it proved too computationally expensive because we would need to interpolate along every edge being tested. As a result, we use discrete time layers by 'snapping' a query time to the nearest sample value.

However, this introduces a subtle failure mode. A configuration that is visible at time  $t$  by the cameras may be incorrectly classified as 'safe' if the nearest sampled time lies just outside the camera’s field of view. This leads the planner to insert incorrect wait commands at nodes that are still partially visible (near the boundary of known camera view), believing that it is safe.

To mitigate this, we tried increasing the number of number of time layers to improve temporal resolution and reduce error, but it increases the cost of computation drastically (takes much longer to build the RRT graph). Second, the option we ended up picking, is to increase our visibility check to evaluate multiple nearby time samples (previous, next) and marking a configuration as visible if it’s detected in any of them. This means that instead of 'snapping' the time to one closest layer, we

check nearby layers (floor and ceiling function, equivalently) because the areas the camera sweeps are large enough that the issue only happens at the borders (where one step is the difference between being seen or not). Both approaches resulted in the same path when we tested it, so we decided to choose the extra visibility checks to reduce computation time.

### 3 Technical Details

#### 3.1 State Representation

The planning problem is formulated in a three-dimensional state space  $(x, y, t)$ , where  $(x, y)$  represents the robot's position in the environment and  $t$  denotes time. Because the cameras rotate periodically with angular velocity  $\omega$ , the system exhibits periodic dynamics with period

$$T = \frac{2\pi}{|\omega|}.$$

Thus, time is treated modulo  $T$ , allowing the planner to operate within a bounded temporal domain while still capturing the full range of camera configurations.

#### 3.2 Node Sampling

Nodes are sampled uniformly in the spatial domain and from a discrete set of time layers within one period:

$$t \in \{t_1, t_2, \dots, t_L\}.$$

Each sampled node is accepted only if it satisfies two conditions:

1. It lies outside all static obstacles.
2. It is not within the field of view of any camera at time  $t$ .

This ensures that all nodes represent valid, safe robot states.

**Spatiotemporal RRT** The planner reasons over a 3D state space,  $(x, y, t)$ . Although the planner reasons over this state, it does not sample time independently when generating new nodes. Instead, random samples are drawn in  $(x, y)$  and the time coordinate is determined by the parent node from which the new node is reached.

$$t_{\text{new}} = (t_{\text{parent}} + \Delta t) \mod T,$$

#### 3.3 Edge Construction and Dynamic Feasibility

Edges between nodes are directed and respect forward time progression. For two nodes  $i$  and  $j$ , the forward time difference is defined as:

$$\Delta t = (t_j - t_i) \mod T.$$

Let  $d_{ij}$  denote the Euclidean distance between the two nodes. Given a constant robot speed  $v$ , the required travel time is:

$$t_{\text{travel}} = \frac{d_{ij}}{v}.$$

An edge is considered dynamically feasible if one of the following holds:

- **Motion edge:**

$$|\Delta t - t_{\text{travel}}| \leq \epsilon,$$

ensuring that the robot can traverse the spatial distance in the allotted time.

- **Wait edge:** If  $d_{ij}$  is below a small threshold, the robot is allowed to remain approximately stationary while time advances.

This formulation enables the planner to coordinate movement with timing, including waiting for safe opportunities to proceed.

**Spatiotemporal RRT** The cost of connecting a parent node to a new node is defined as the sum of waiting time and travel time. Each node stores a cost-to-come value representing the total elapsed time required to reach it from the start. Consequently, the planner does not optimize purely for shortest spatial distance. A shorter geometric path may require substantial waiting if it passes through frequently observed regions, whereas a slightly longer path may have lower overall cost if it can be traversed with little or no delay. The objective is therefore to minimize total elapsed traversal time rather than path length alone.

$$c_{\text{edge}} = t_{\text{wait}} + \frac{d}{v},$$

### 3.4 Continuous Safety Verification

For each candidate edge, safety is verified through continuous interpolation in  $(x, y, t)$ . The edge is discretized into a sequence of intermediate states:

$$(x(s), y(s), t(s)), \quad s \in [0, 1].$$

At each sampled point along the edge:

- The position must remain outside all obstacles.
- The robot must not lie within any camera’s field of view at time  $t(s)$ .

This ensures that the entire trajectory segment, not just the endpoints, is safe from both collisions and detection.

### 3.5 Spatiotemporal RRT: Parent Selection, Rerouting

After evaluating all candidate parents, the planner selects the one that yields the total minimum elapsed-time cost. This is an analog of choosing the lowest-distance cost among nearby nodes. Once the new node is inserted, the planner performs a rewiring step over neighboring nodes. For each neighboring node, it checks whether reaching the new node would produce a lower cost. If so, the neighboring node is reassigned. Effectively, when a new node is added, it is placed in a ”search radius” of how the robot can get there from its current options (branches) of its exploration.

$$x_{\text{parent}} = \arg \min_{x \in \mathcal{N}(x_{\text{new}})} [c_{\text{reach}}(x) + c_{\text{edge}}(x, x_{\text{new}})],$$

A neighboring node  $x_{\text{neighbor}}$  is rewired if:

$$c_{\text{reach}}(x_{\text{new}}) + c_{\text{edge}}(x_{\text{new}}, x_{\text{neighbor}}) < c_{\text{reach}}(x_{\text{neighbor}}).$$

### 3.6 Camera Visibility Model

Each camera is modeled as a rotating sensor with a fixed field of view and maximum range. The orientation of a camera at time  $t$  is given by:

$$\theta(t) = \theta_0 + \omega t.$$

From this orientation, a visibility polygon is computed that represents the region observable by the camera, accounting for occlusion by walls. A state is considered detected if it lies within any camera’s visibility polygon at the corresponding time.

### 3.7 PRM Graph Search

After constructing the roadmap, the PRM planning problem is solved using A\* search. Multiple goal nodes are instantiated at different time values to account for periodicity. The cost of traversing an edge is defined as the elapsed time  $\Delta t$ , and the heuristic function is:

$$h(x, y) = \frac{\sqrt{(x - x_g)^2 + (y - y_g)^2}}{v},$$

which represents the minimum possible time to reach the goal under straight-line motion. This heuristic is admissible and guides the search efficiently toward time-optimal solutions.

### 3.8 RRT\*

Unlike the PRM approach, the RRT\* planner does not construct a graph followed by a separate search phase. Instead, it incrementally builds a tree rooted at the start node, directly optimizing the path during construction.

A path to the goal is obtained when a newly added node can be connected to the goal region via a dynamically feasible and safe trajectory. The resulting solution corresponds to the lowest-cost path discovered during tree construction.

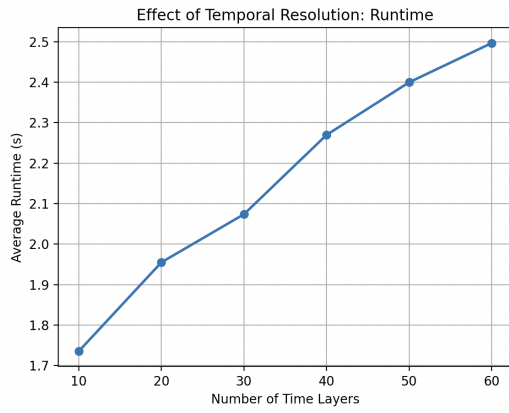
## 4 Results

We conducted parameter analysis on each approach to determine the effects of different variables and find the most feasible approach.

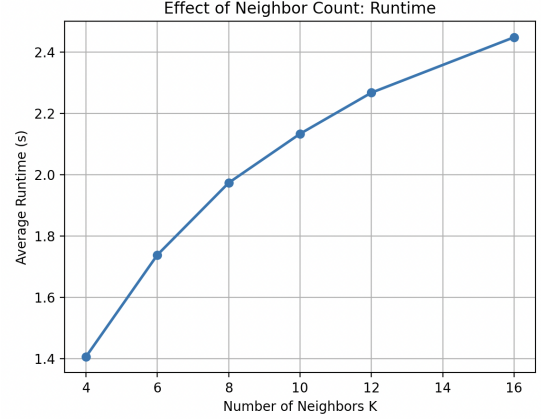
### 4.1 PRM

Increasing the number of time layers results in a consistent increase in runtime. This is expected because a higher temporal resolution expands the size of the state space, effectively increasing the number of nodes in the roadmap. Additionally, more time layers require recomputation of camera visibility and increase the number of candidate edges that must be validated. As a result, both graph construction and search become more computationally expensive. This demonstrates a clear trade-off: finer temporal resolution improves the planner’s ability to capture short visibility gaps, but comes at a higher computational cost. The approximately linear trend suggests that temporal discretization is a dominant factor in scaling runtime.

Increasing the number of nearest neighbors,  $K$ , leads to higher runtime due to the increased number of edges in the roadmap. Each additional neighbor requires edge validation, including dynamic feasibility checks and continuous collision and visibility verification. As a result, graph



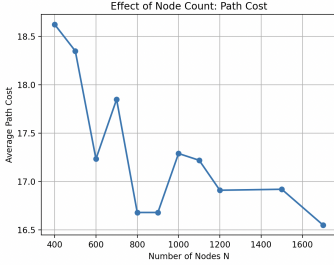
(a) Temporal resolution vs runtime



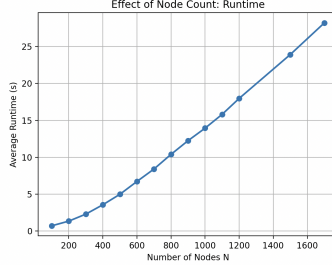
(b) Neighbor count vs runtime

Figure 1: Effect of temporal discretizations and neighbor count on PRM runtime

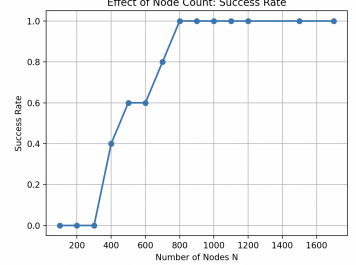
construction becomes significantly more expensive as  $K$  grows. The rate of increase begins to taper at higher values of  $K$ , suggesting diminishing returns in connectivity relative to computational cost. While larger  $K$  improves graph connectivity, the added edges become increasingly redundant, contributing more to computation time than to solution quality.



(a) Node count vs path cost



(b) Node count vs runtime



(c) Node count vs success rate

Figure 2: Node count on path cost, runtime, and success rate

Increasing the number of nodes generally leads to lower path cost, as a denser roadmap provides more candidate connections and allows the planner to find more efficient trajectories. However, the path cost does not decrease monotonically and exhibits some variability due to the stochastic nature of sampling. After a certain point, improvements become marginal, indicating diminishing returns in solution quality despite increased sampling density.

Runtime increases significantly as the number of nodes grows, reflecting the increased computational cost of constructing and searching the roadmap. A larger number of nodes results in more candidate edges, each requiring dynamic feasibility checks and continuous collision and visibility verification. This leads to a rapid growth in computation time, especially at higher node counts.

As the number of sampled nodes increases, the success rate improves significantly. At low node counts, the roadmap is too sparse to capture feasible spatiotemporal connections, particularly in regions where safe paths require precise timing. As  $N$  increases, the probability of sampling valid states and connecting them into a feasible path improves. Beyond approximately 800 nodes, the success rate reaches 1 and remains stable, indicating that the roadmap has sufficient coverage of

the state space.

Runtime increases significantly as the number of nodes grows, reflecting the increased computational cost of constructing and searching the roadmap. A larger number of nodes results in more candidate edges, each requiring dynamic feasibility checks and continuous collision and visibility verification. This leads to a rapid growth in computation time, especially at higher node counts. We chose to continue with 800 nodes as this allowed for enough density to find a solution and also showed low path cost.

## 4.2 RRT Temporal

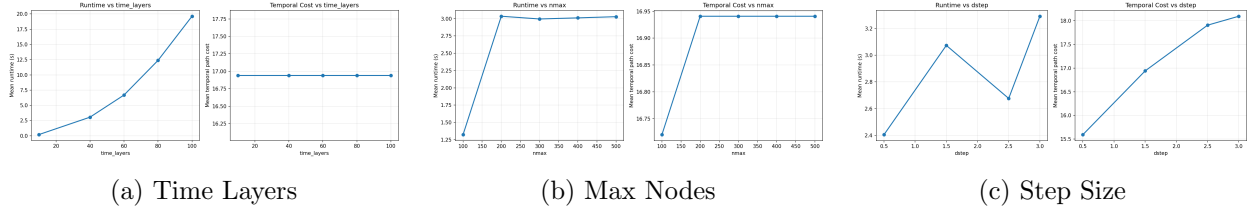


Figure 3: Temporal RRT\* planner under varying parameters.

In graph (a), we see that increasing time layers has an impact on runtime but minimal effect on path cost. Temporal discretization impacts the cost of visibility checking, thus increasing the cost of post-processing. A larger number of layers means more times are evaluated when assigning the time layer after building the tree to the goal. However, since it is applied only after the tree is built, increasing temporal resolution doesn't affect path quality. This is a limitation of the RRT approach, because the RRT path is generated in space, increasing time resolution doesn't change the quality of the path.

In graph (b) increasing the maximum number of nodes leads to a moderate increase in runtime, which caps beyond a certain threshold. This is because once the RRT has sufficient coverage of the space, additional nodes provide little difference in exploration quality (runtime and temporal cost). So for the RRT\*, we choose not to text against nmax.

In graph (c), increasing step size generally reduces runtime because the tree can expand more aggressively and require fewer layers in the tree to reach the goal. However, this comes at the cost of increased path cost, as larger steps produce less precise graphs and reduce the planner's ability to navigate around obstacles/hide behind barriers.

## 4.3 Spatiotemporal RRT

Because the map we chose is inherently solvable, we do not include success rate as a metric. However, we do run each set of parameters 20 times, and we see there is some high variance in the metrics for some cases. Runtime increases with finer temporal discretization (more layers) and larger neighborhood radius, as expected, while larger step sizes reduces runtime but decreases path quality (increasing path cost). Path cost decreases with increasing neighborhood radius, because each sampled node has more parent candidates to decide from. The number of nodes decreases with larger step sizes because the path can more aggressively expand and add nodes in space. Increasing the step size reduces path quality because the planner makes larger, coarse extensions that cannot adapt well to obstacles and visibility constraints. This causes the path to overshoot safe regions and results in less precise trajectories. Smaller step sizes allow better adjustments, producing smoother, more 'direct' paths at the cost of increased computation. This method is also susceptible to RRT

tradeoffs, such as if we increase the sampling in the direction of the goal too much, it won't be able to find a feasible path avoiding the cameras because sometimes it must move slightly away from the goal (i.e. to duck behind a barrier) in order to stealthily cross the maze.

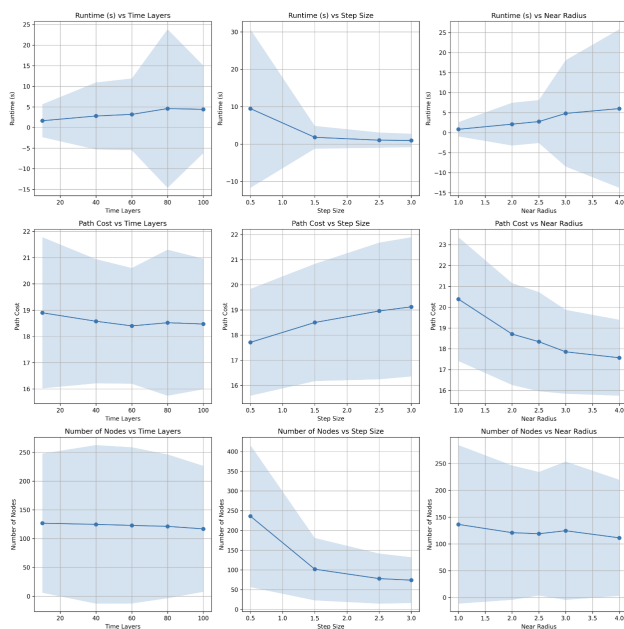


Figure 4: Spatialtemporal RRT with parameter sweeps.

## 5 Comparisons

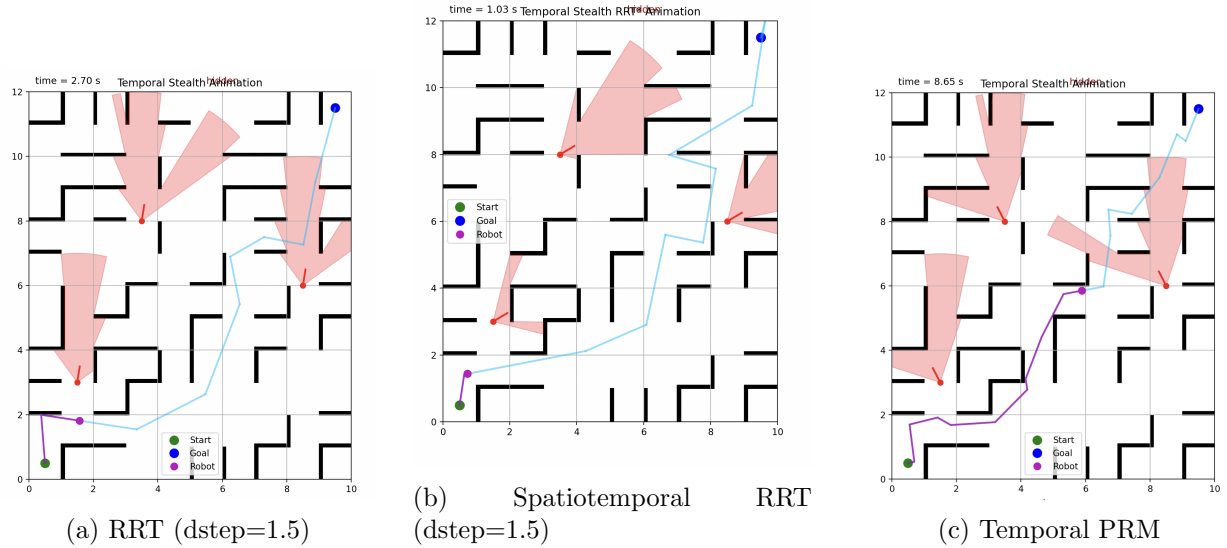


Figure 5: Comparison of paths generated by RRT with post-processing, spatiotemporal RRT, and temporal PRM.

Each planner approaches the temporal problem in a unique way. This results in varying performance in the same map or maze, creating certain advantages or shortcomings for each.

The RRT with post-processing generates a largely spatial graph that prioritizes geometric efficiency, resulting in a somewhat jagged trajectory towards the goal. However, because temporal constraints are handled only after the path is constructed, the resulting trajectory may pass through regions that are frequently visible to cameras, requiring precise timing or waiting to remain undetected. A further analysis of the RRT with post-processing shows that step size is the dominant factor affecting path cost as larger steps produce worse geometric paths that are harder to schedule around cameras. But this sensitivity is indirect as the planner has no awareness during tree construction that a given segment will be difficult to time. The spatiotemporal RRT and PRM both incorporate this awareness during planning, so their path costs are less affected to geometric path quality.

The spatiotemporal RRT explicitly incorporates time during tree expansion, leading to a path that actively avoids camera visibility regions. The resulting trajectory is less direct and exhibits more deliberate detours, but these deviations reflect alignment with safe time windows rather than purely spatial considerations. As a result, the path is inherently feasible without requiring significant post-processing adjustments.

The temporal PRM constructs a roadmap over the full spatiotemporal space before performing search. This allows it to capture a richer set of feasible connections between states. As a result, the PRM produces smoother and more structured trajectories that better coordinate spatial motion with timing. The key difference is that PRM separates sampling from search, whereas RRT combines them. By precomputing a roadmap, PRM captures global connectivity in the spatiotemporal space, allowing the planner to reason about multiple possible routes and timings simultaneously. In contrast, RRT commits to a single exploratory tree, which can lead to suboptimal or less coordinated solutions. PRM performs well when multiple queries are required, as the same roadmap can be reused without reconstruction. However, this comes at the cost of higher upfront computation

and memory usage compared to RRT.

Overall the 3 approaches depend on the application constraints. If minimizing the total time is the priority and is computationally feasible, then the spatiotemporal seems to be the most-time optimal path by considering both the geometry and the timing. The PRM provides great reliability and low variance, so it performs well when safety matters most (i.e avoiding detection). Lastly, the RRT with post-processing is best suited when a fast and simplistic is required and the environment has low camera density.